National University of Computer and Emerging Sciences

AL-2002: Artificial Intelligence
Lab Manual 12

Department of Computer Science
FAST-NU, Lahore, Pakistan

Table of Contents

Objectives

After performing this lab, students shall be able to:

- ✔ Understand the working of Convolutional Neural Networks (CNNs).
- ✔ Identify and explain different CNN layers (Convolution, Pooling, Fully Connected).
- ✔ Implement a CNN model using Python and deep learning libraries.
- ✔ Train and evaluate CNN models on image datasets.
- ✔ Interpret results using accuracy and loss curves.

1 Introduction to CNNs

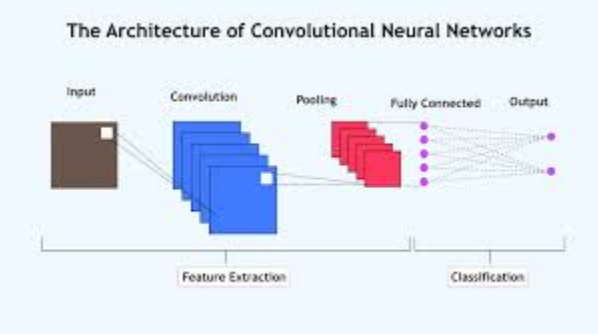
A Convolutional Neural Network (CNN) is a type of deep learning model primarily used for processing structured grid data such as images. CNNs automatically learn spatial hierarchies of features using convolution operations.

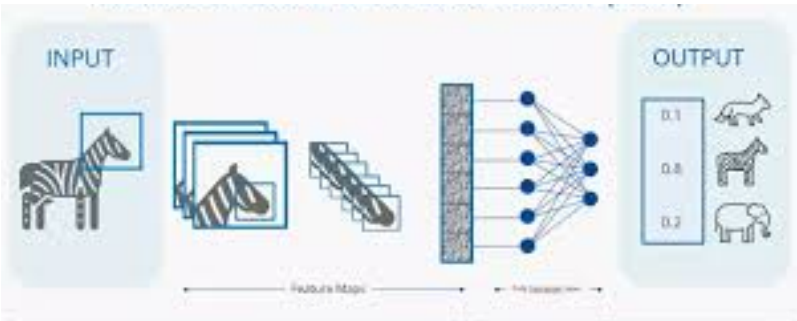
Unlike traditional neural networks, CNNs require less preprocessing and are highly effective in image classification, object detection, and pattern recognition tasks.

Images contain spatial information (height, width, channels). CNNs

- Preserve spatial relationships
- Require fewer parameters than fully connected networks
- Automatically extract features (edges, textures, shapes)

A typical CNN consists of multiple layers stacked together:





1.1 Input Layer

The input layer defines the shape of the data entering the network. For image data, this includes height, width, and number of channels.

For example, grayscale images have 1 channel, while RGB images have 3 channels.

```
tf.keras.Input(shape=(28, 28, 1))
```

shape=(28,28,1) means 28 = height, 28 = width, 1 = grayscale channel

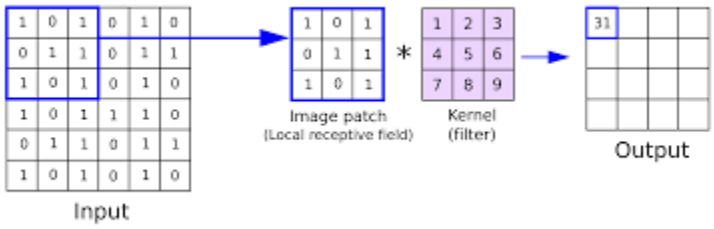
1.2 Convolution Layer

The convolution layer applies filters (kernels) to extract features from input images.

Mathematically:

$$Feature\ Map = Input * Kernel$$

Where: Input = Image matrix, Kernel = Filter matrix



```
tf.keras.layers.Conv2D(32, (3,3), activation='relu')
```

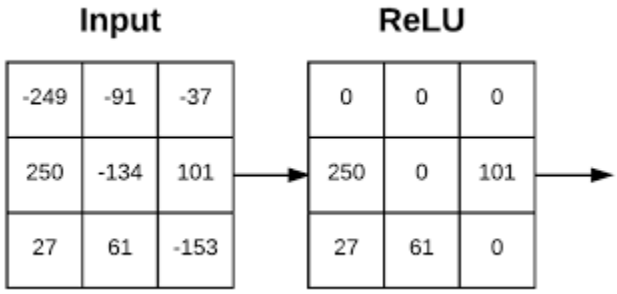
- **32 (filters):** number of feature detectors
- **(3,3)**: size of filter (kernel)
- **activation='relu':** introduces non-linearity

1.3 Activation Function (ReLU)

ReLU (Rectified Linear Unit) introduces non-linearity:

$$f(x) = max\ (0,x)$$

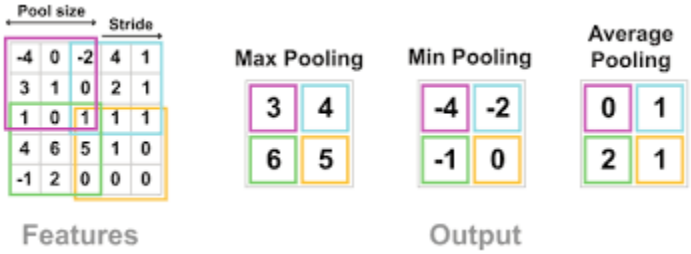
It helps the network learn complex patterns.



1.4 Pooling Layer

Pooling reduces spatial dimensions and computation. Types:

- Max Pooling (most common)
- Average Pooling



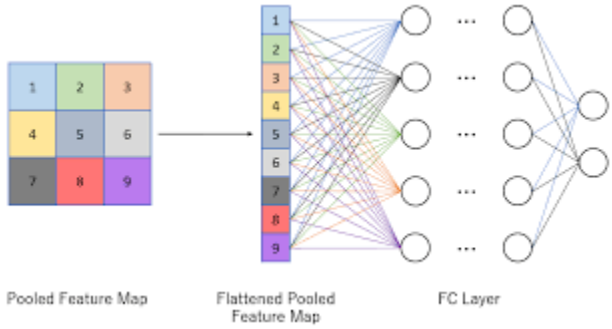

```
tf.keras.layers.MaxPooling2D((2,2))
```

pool_size: Size of pooling window (2×2 reduces dimensions by half)

1.5 Flatten Layer

Before passing data into dense layers, the 2D feature maps must be converted into a 1D vector.

```
tf.keras.layers.Flatten()
```



1.6 Fully Connected (Dense) Layer

Dense layers learn complex relationships between extracted features and output labels.

The final layer usually uses **softmax** activation for multi-class classification.

```
tf.keras.layers.Dense(64, activation='relu')
tf.keras.layers.Dense(10, activation='softmax')
```

2 Implementing CNN using Python

We will use a sample dataset such as handwritten digits (MNIST).

First, the dataset (MNIST) is loaded, which contains grayscale images of handwritten digits. The images are reshaped into a 4D format (samples, height, width, channels) because CNNs expect channel information.

```
import tensorflow as tf
from tensorflow.keras import datasets
```

```
# Load dataset
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()

# Reshape for CNN
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)
```

Next, a Sequential model is created, which allows stacking layers one after another. A Rescaling layer is added inside the model after input to automatically normalize pixel values from [0–255] to [0–1]. The first convolution layer applies 32 filters of size 3×3 to extract basic features from the image. This is followed by a max pooling layer that reduces the spatial dimensions, making computation efficient.

A second convolution layer with 64 filters is added to capture more complex patterns, followed again by pooling. The output is then flattened into a one-dimensional vector so it can be fed into dense layers.

A dense layer with 64 neurons learns high-level patterns, and the final dense layer with 10 neurons (one for each digit class) uses softmax activation to output probabilities.

```
model = tf.keras.Sequential([

    # Input + Normalization Layer
    tf.keras.layers.Rescaling(1./255, input_shape=(28, 28, 1)),

    # Convolution + Pooling
    tf.keras.layers.Conv2D(32, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2,2)),

    # Second Convolution + Pooling
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2,2)),

    # Flatten
    tf.keras.layers.Flatten(),

    # Fully Connected Layers
    tf.keras.layers.Dense(64, activation='relu'),
```

```
    tf.keras.layers.Dense(10, activation='softmax')    # Output Layer
])

model.summary()
```

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 64)	102,464
dense_1 (Dense)	(None, 10)	650

Total params: 121,930 (476.29 KB)

Trainable params: 121,930 (476.29 KB)

Non-trainable params: 0 (0.00 B)

The model is compiled using the Adam optimizer and a suitable loss function for classification. It is then trained for multiple epochs, during which it learns patterns in the data.

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',# for multiclass
              metrics=['accuracy'])

history = model.fit(x_train, y_train, epochs=5, validation_split=0.2)
```

Finally, the model is evaluated on test data to measure its accuracy (you can find other evaluation metrics as well in the same manner we covered under ANN (Tensorflow-Lab 10).

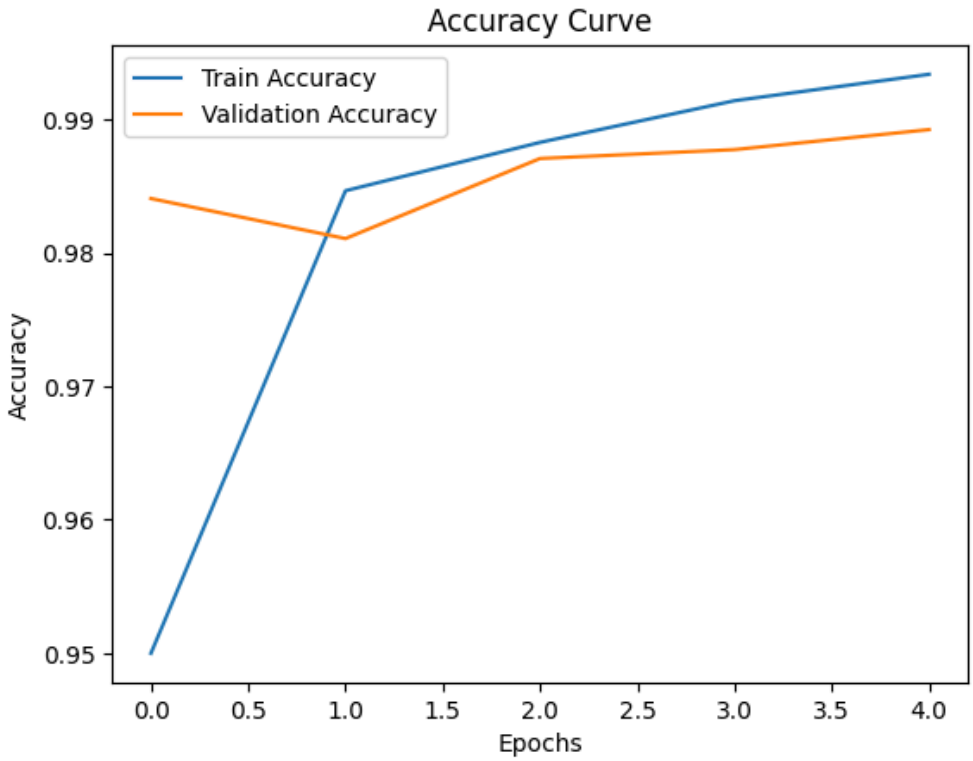
```
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test Accuracy:", test_acc)
```

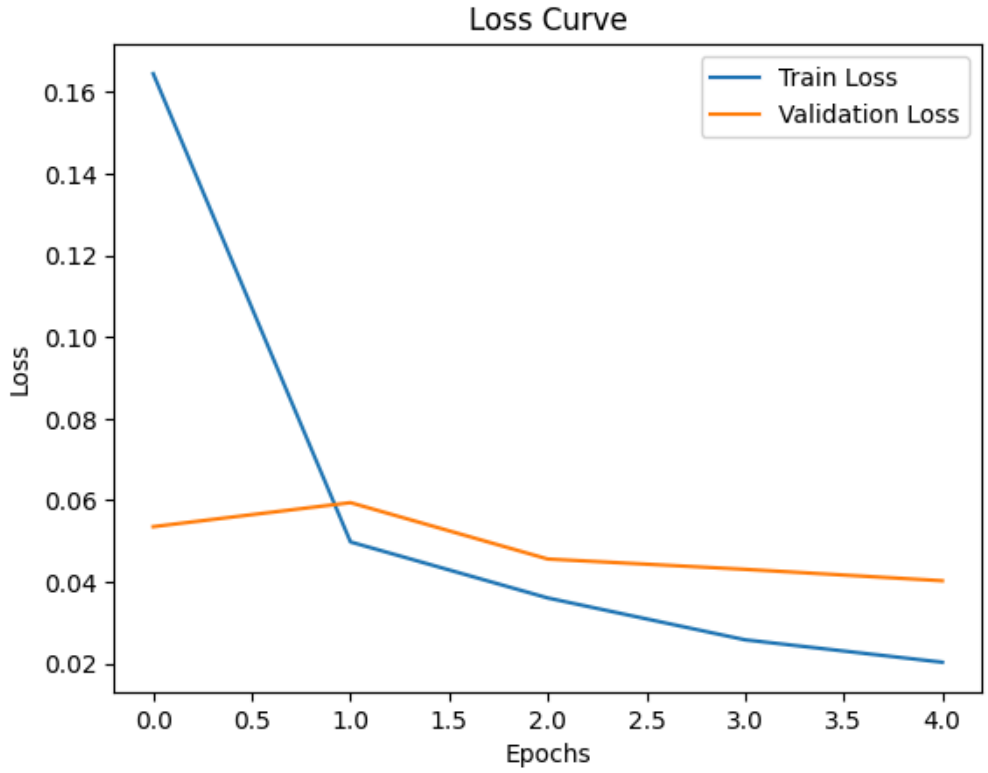
Visualization of Training

```
import matplotlib.pyplot as plt

# Accuracy Plot
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title("Accuracy Curve")
plt.show()

# Loss Plot
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Loss Curve")
plt.show()
```


Accuracy Curve
Train Accuracy
Validation Accuracy
Accuracy
0.99
0.98
0.97
0.96
0.95
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Epochs


Loss Curve
Train Loss
Validation Loss
Loss
0.16
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Epochs